

NAME: S.DEVI SHREE

REG.NO: 192511149

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO3: Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

1.Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price).Identify FDs and normalize to 3NF.

Online Shopping: Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price) → 3NF

Candidate key: OrderID (assume one product line per order row)

Functional Dependencies (FDs):

- OrderID → CustID, ProdID, Qty
- CustID → CustName (transitive, via OrderID)
- ProdID → ProdName, Price (transitive, via OrderID)

Analysis: The key is a single attribute, so no partial dependency exists (1NF and 2NF are automatically satisfied). But CustName and ProdName, Price don't depend on OrderID directly — they depend on CustID and ProdID, which are themselves non-key attributes. This is a transitive dependency, which violates 3NF.

Decomposition to 3NF:

Table	Attributes	Key
Orders	OrderID, CustID, ProdID, Qty	OrderID
Customer	CustID, CustName	CustID
Product	ProdID, ProdName, Price	ProdID

No non-key attribute now depends transitively on the key → 3NF achieved.

2.A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Hospital: Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)
→ BCNF

Candidate key: PID

FDs:

- PID → PName, DocID, WardNo
- DocID → DocName, Specialization
- WardNo → WardName

Anomalies present in the unnormalized table:

- **Insertion:** Can't record a doctor's specialization until a patient is assigned to them.
- **Update:** If a doctor's name changes, it must be updated in every row of every patient under that doctor — risk of inconsistency.
- **Deletion:** If the only patient under a doctor is discharged (row deleted), all information about that doctor is lost.

Decomposition:

Table	Attributes	Key
Patient	PID, PName, DocID, WardNo	PID
Doctor	DocID, DocName, Specialization	DocID
Ward	WardNo, WardName	WardNo

BCNF check: In every resulting table, the only determinant is the primary key itself (e.g., in Doctor, only DocID determines anything). Since every determinant is a candidate key → **BCNF satisfied.**

3.Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

University: Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade) → 2NF

Candidate key: (SID, CourseID) — composite, since a student takes many courses and a course has many students.

FDs:

- SID → SName (partial — depends only on SID, part of the key)
- CourseID → CourseName, Faculty (partial — depends only on CourseID)

- $SID, CourseID \rightarrow Grade$ (full dependency — depends on the whole key)

Analysis: SName and CourseName, Faculty each depend on only *part* of the composite key, not the whole key $\rightarrow$ violates **2NF**.

Decomposition to 2NF:

Table	Attributes	Key
Student	SID, SName	SID
Course	CourseID, CourseName, Faculty	CourseID
Enrollment	SID, CourseID, Grade	(SID, CourseID)

Every non-key attribute now depends on the *entire* key of its table $\rightarrow$ **2NF achieved** (this also happens to be 3NF, since no transitive dependency remains).

4. Given the airline booking case: **Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)**. Normalize up to BCNF.

Airline: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date) $\rightarrow$ BCNF

Candidate keys: (FlightNo, Date, Seat) and (FlightNo, Date, PassID) — both uniquely identify a booking.

FDs:

- $FlightNo \rightarrow DeptCity, ArrCity$
- $PassID \rightarrow PassName$
- $FlightNo, Date, Seat \rightarrow PassID$
- $FlightNo, Date, PassID \rightarrow Seat$

Analysis: $FlightNo \rightarrow DeptCity, ArrCity$ is a problem: FlightNo alone is **not** a candidate key (it's only part of one), yet it determines other attributes. This violates **BCNF** (and even 2NF, since it's a partial dependency on a composite key).

Decomposition to BCNF:

Table	Attributes	Key
Flight	FlightNo, DeptCity, ArrCity	FlightNo
Passenger	PassID, PassName	PassID
Booking	FlightNo, Date, PassID, Seat	(FlightNo, Date, PassID) <i>or</i> (FlightNo, Date, Seat)

Now in every table, every determinant is a candidate key $\rightarrow$ **BCNF satisfied**.

5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

Multinational HR System → 3NF

Assumed schema: Employee(EmpID, EmpName, DeptID, DeptName, DeptLocation, Salary, CountryCode, CountryName)

Candidate key: EmpID

FDs:

- EmpID → EmpName, DeptID, Salary, CountryCode
- DeptID → DeptName, DeptLocation (transitive)
- CountryCode → CountryName (transitive)

Transitive dependency chains:

EmpID → DeptID → DeptName, DeptLocation

EmpID → CountryCode → CountryName

Decomposition to 3NF:

Table	Attributes	Key
Employee	EmpID, EmpName, DeptID, Salary, CountryCode	EmpID
Department	DeptID, DeptName, DeptLocation	DeptID
Country	CountryCode, CountryName	CountryCode

No non-key attribute depends on another non-key attribute → **3NF achieved.**

6. Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Library: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate) → 4NF

FDs (single-valued):

- MemberID → MemberName
- BookID → Title

Multivalued Dependencies (MVDs): A book can have **multiple authors** and belong to **multiple genres**, and these two facts are *independent* of each other:

- BookID →→ Author
- BookID →→ Genre

Why this is a problem: If a book has 2 authors and 3 genres, storing them together forces **6 rows** (2×3 cross-product) just to represent facts that are logically independent — a classic 4NF violation (redundancy without an FD to explain it).

Decomposition to 4NF:

Table	Attributes	Key
Member	MemberID, MemberName	MemberID
Book	BookID, Title	BookID
BookAuthor	BookID, Author	(BookID, Author)
BookGenre	BookID, Genre	(BookID, Genre)
Borrow	MemberID, BookID, BorrowDate	(MemberID, BookID, BorrowDate)

Each independent multivalued fact is isolated into its own relation → **4NF achieved**.

7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Telecom Billing: Identify Keys and FDs (no decomposition required)

Assumed schema: Billing(PhoneNo, CustID, CustName, PlanID, PlanName, PlanCost, BillMonth, CallDuration, Amount)

Candidate key: (PhoneNo, BillMonth) — one bill is generated per phone number per month.

Primary key: (PhoneNo, BillMonth)

Functional Dependencies:

- PhoneNo → CustID, PlanID
- CustID → CustName
- PlanID → PlanName, PlanCost
- PhoneNo, BillMonth → CallDuration, Amount

Analysis (BL2 — understand only): CustName depends transitively on PhoneNo through CustID, and PlanName, PlanCost depend transitively through PlanID. PhoneNo alone is also a **partial determinant** relative to the composite key (PhoneNo, BillMonth) — this identifies where 2NF/3NF violations *would* arise if asked to normalize.

8. A retail inventory system has Product (PID, PName, SupplierID, Supplier Name, Category, Price, Warehouse). Apply complete normalization.

Retail Inventory: Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse) → Full Normalization

Candidate key: PID

FDs:

- PID → PName, SupplierID, Category, Price, Warehouse
- SupplierID → SupplierName (transitive)

1NF: Already atomic — satisfied.

2NF: Key is a single attribute (PID), so no partial dependency is possible — satisfied automatically.

3NF violation: SupplierName depends on SupplierID, not directly on PID.

Decomposition:

Table	Attributes	Key
Product	PID, PName, SupplierID, Category, Price, Warehouse	PID
Supplier	SupplierID, SupplierName	SupplierID

BCNF check: In both tables the only determinant is the primary key → this schema is simultaneously in **3NF and BCNF**.

9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

School ERP: Poorly Normalized Schema → BCNF (with justification)

Assumed schema: SchoolRecord(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, Subject, Marks)

Candidate key: (StudentID, Subject)

FDs:

- StudentID → StudentName, ClassID
- ClassID → ClassName, TeacherID
- TeacherID → TeacherName, Subject
- StudentID, Subject → Marks

Anomalies: Update anomaly (class/teacher name repeated per student and must be changed everywhere), insertion anomaly (can't add a new class with no students yet), deletion anomaly (removing the last student of a class erases the class/teacher record).

Justification for decomposition: StudentID, ClassID, and TeacherID are all determinants that are **not** the full candidate key — each causes a partial or transitive dependency.

Decomposition:

Table	Attributes	Key
Student	StudentID, StudentName, ClassID	StudentID
Class	ClassID, ClassName, TeacherID	ClassID
Teacher	TeacherID, TeacherName, Subject	TeacherID
Marks	StudentID, Subject, Marks	(StudentID, Subject)

Every determinant in every resulting table is now a candidate key of that table → **BCNF satisfied.**

10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Movie Streaming: Join Dependency → 5NF (with lossless-join verification)

Assumed schema: Streaming(Movie, Actor, Platform)

Business rule: This is a genuine **three-way relationship**, not decomposable into two independent binary relationships — a specific (Movie, Actor, Platform) combination is valid *only if*:

(Movie, Actor) exists in the cast list, **and** (Movie, Platform) exists in the licensing list, **and** (Actor, Platform) exists in the actor's platform-exclusivity list.

This is a **Join Dependency**: $JD(R_1, R_2, R_3)$ where none of the three pairwise projections alone (nor any two of them) can reconstruct the original relation losslessly — you need **all three** to rejoin without generating spurious rows. This distinguishes it from an MVD case (which needs only two projections).

Decomposition to 5NF:

Table	Attributes
R1 – MovieActor	Movie, Actor
R2 – MoviePlatform	Movie, Platform
R3 – ActorPlatform	Actor, Platform

Lossless-join verification (example):

Suppose the original Streaming relation contains exactly:
(Inception, Leonardo, Netflix), (Inception, Cillian, Netflix)

Projecting into R1, R2, R3 and then computing $R1 \bowtie R2 \bowtie R3$:

- $R1 = \{(Inception, Leonardo), (Inception, Cillian)\}$
- $R2 = \{(Inception, Netflix)\}$
- $R3 = \{(Leonardo, Netflix), (Cillian, Netflix)\}$

Rejoining $R1 \bowtie R2$ (on Movie) $\bowtie R3$ (on Actor, Platform) reproduces **exactly** the original 2 tuples — no spurious rows are introduced. This confirms $JD(R1, R2, R3)$ holds and the decomposition is **lossless**, satisfying **5NF** (no further decomposition is possible without loss).